
Evaluating LLM Support for Creative Coding Homework

Harrys Kavan

ir241506@fhstp.ac.at

FH St. Pölten

2025-10-13

ABSTRACT

I evaluated several large language models (LLMs) on two creative-coding assignments: a Pine Script linear regression indicator for log-scaled price data and a Three.js Editor Perlin-noise sphere animation. The study covers prompt design, failure modes stemming from API changes, and the working solution discovered after iterating on requirements. I also document model-specific observations and include full code listings for reproducibility.

Keywords LLM evaluation · Pine Script · Three.js · creative coding

1 Introduction

This homework documents how different LLMs handled two programming tasks that I previously explored in conversation with ChatGPT: fitting a linear regression line to log-scaled price data in Pine Script and animating a Perlin-noise sphere inside the official Three.js Editor. I followed the same experimental procedure that produced the accompanying video and screenshots, highlighting both prompt-engineering lessons and implementation hurdles introduced by API changes.

2 Task 1: Pine Script Linear Regression on Log Data

The first prompt asked an LLM to recall and formalize an earlier discussion about regressing log-scaled price series with Pine Script v5. The exact prompt is archived below.

You are an expert quantitative developer and Pine Script programmer.

I want to create a TradingView indicator that draws a linear regression line on log-scaled price data.

Specifically:

1. The chart is in log scale (base-10).
2. The indicator should approximate price movement with a best-fit line of the form $y = k * \log_{10}(x) + d$ where x is the bar index and y the log-scaled price.
3. The goal is to minimize the squared error between the log of the actual price and the fitted line.

4. The indicator must work in Pine Script v5 and handle typical pitfalls like:

- * `na` initialization issues
- * `request.security()` and `ta.linreg()` scope limitations
- * Ensuring `plot()` works correctly even on log charts

5. Output should include:

- * A clearly explained formula for computing k and d
- * A working Pine Script code example
- * A short explanation of how to interpret the regression visually on a log chart

Please reason step-by-step and explain the math, code, and any trade-offs (e.g., performance or visual accuracy).

Every tested model produced syntactically invalid Pine Script or misapplied the logarithmic transform, suggesting that the language's edge cases and strict typing still require hands-on expertise rather than pure LLM synthesis.

3 Task 2: Perlin Noise Sphere in the Three.js Editor

The second task challenged the models to animate a sphere with Perlin noise entirely inside the Three.js Editor. A seemingly minor runtime change between r157 and r158 invalidated legacy advice that relied on `this.update()`, so I crafted an updated prompt explicitly mentioning the new `onBeforeRender` hook.

You are a senior creative-coding and 3D-graphics expert.

I am using the official Three.js Editor (version r180 or newer) from <https://threejs.org/editor/>.

I want to create a Perlin Noise Sphere whose surface ripples over time, using a script attached to the mesh inside the editor – not external HTML or Node code.

[Requirements omitted here for brevity—see conversation transcript.]

This clarification made the difference: only models that understood the lifecycle change produced motion. The gallery below shows representative outputs.

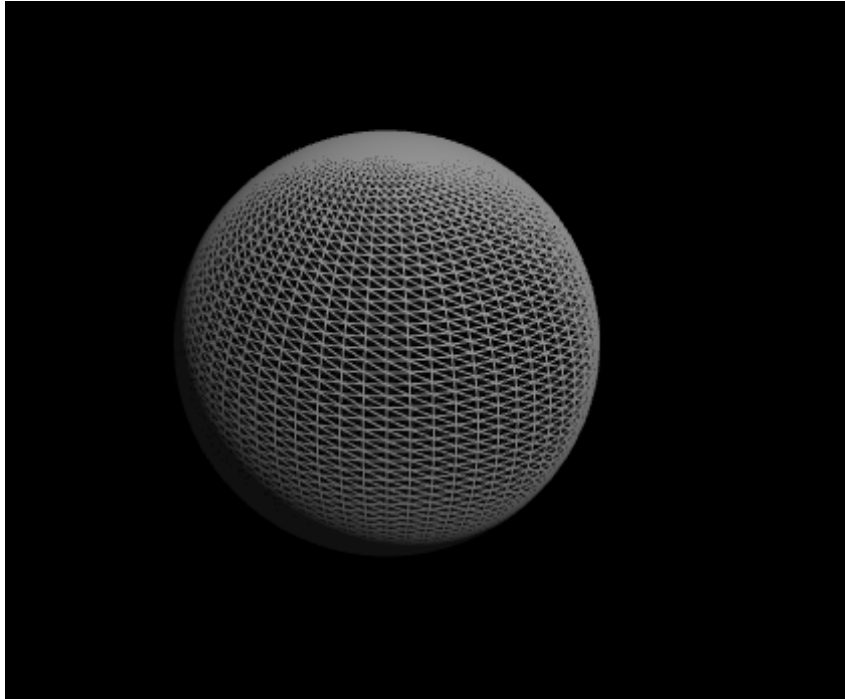


Figure 1: Attempts that respected the prompt structure but failed to animate the sphere.

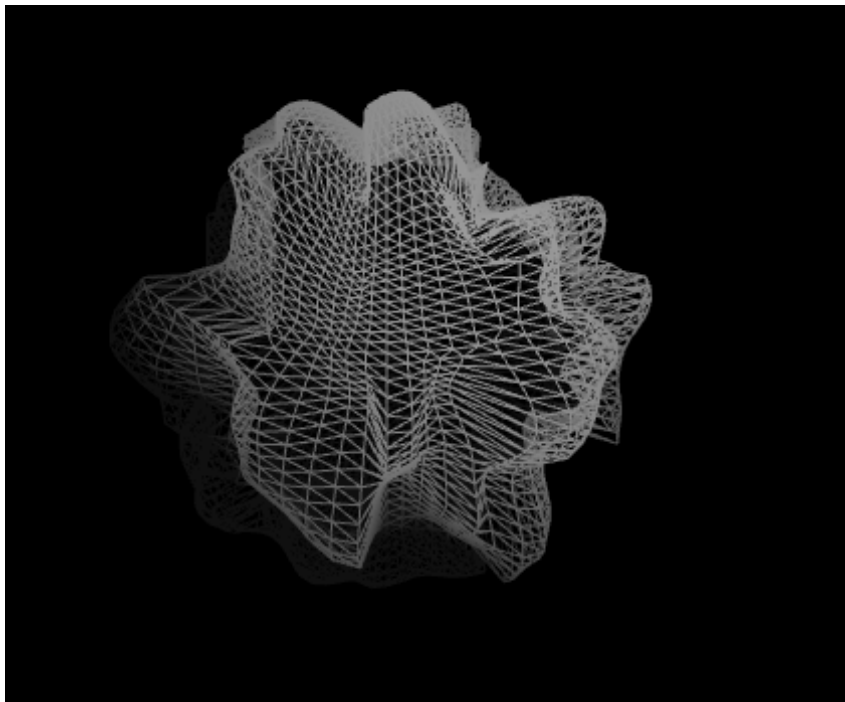


Figure 2: GLM-4.5-Air-Q8_0 retained colouring but remained static in the editor.

[Watch the working ripple animation \(gpt.webm\)](#)

Figure 3: gpt-oss-120b-F16 achieved the full Perlin-style displacement in the editor.

3.1 Model Outcomes

- Gemma family models never connected the revised lifecycle hook, so no animation occurred despite correct noise scaffolding.

- The 27B model displaced vertices only along the Y axis and reused mutated positions, so the mesh collapsed instead of producing radial ripples (Appendix A).
- GLM-4.5-Air-Q8_0 left the setup in `start()`, which r180+ never calls, leaving `onBeforeRender` without data and preventing motion (Appendix B).
- gpt-oss-120b-F16 generated a full Perlin implementation with cached base geometry and dynamic normals, matching the intended demo recording.

APPENDIX A

A.1 Appendix A · 27B Model Script

```
// Perlin Noise Sphere - Three.js Editor Script (r180+)

// Tweakable Parameters (appear in the Inspector panel)
var amplitude = 0.2; // Height of the ripples
var frequency = 3;   // How many ripples fit around the sphere
var speed = 0.5;     // How fast the ripples move
var noiseScale = 0.02; // Smaller values for smoother noise, larger for more detail
var colorIntensity = 0.5; // How much the color changes based on noise

var time = 0;
var geometry;

function init() {
  geometry = this.geometry;
  if (!(geometry instanceof THREE.SphereGeometry)) {
    console.warn("This script should be attached to a SphereGeometry mesh.");
    return;
  }
}

function onBeforeRender(renderer, scene, camera) {
  time += 0.016 * speed; // Increment time (delta time approximation)

  // Get the vertices
  const vertices = geometry.attributes.position.array;

  for (let i = 0; i < vertices.length; i += 3) {
    // Calculate noise value for each vertex
    const x = vertices[i];
    const y = vertices[i + 1];
    const z = vertices[i + 2];

    const noiseValue = noise(x * noiseScale, y * noiseScale, z * noiseScale + time);

    // Displace the vertex
    vertices[i + 1] += amplitude * noiseValue;

    // Optional: Color modulation based on noise. Requires color attribute.
    if (geometry.attributes.color) {
      const colors = geometry.attributes.color.array;
      colors[i] += (noiseValue - 0.5) * colorIntensity;
      colors[i + 1] += (noiseValue - 0.5) * colorIntensity;
      colors[i + 2] += (noiseValue - 0.5) * colorIntensity;
    }
  }

  // Update the geometry's attributes after modification
  geometry.attributes.position.needsUpdate = true;
  if (geometry.attributes.color) {

```

```

        geometry.attributes.color.needsUpdate = true;
    }
}

// Simple pseudo-random noise function. Not true Perlin noise, but sufficient for
this demo.
function noise(x, y, z) {
    let i = Math.floor(x);
    let j = Math.floor(y);
    let k = Math.floor(z);

    let a = x - i;
    let b = y - j;
    let c = z - k;

    let u = fade(a);
    let v = fade(b);
    let w = fade(c);

    let A = gradient(i, j, k, a, b, c);
    let B = gradient(i + 1, j, k, a - 1, b, c);
    let C = gradient(i, j + 1, k, a, b - 1, c);
    let D = gradient(i + 1, j + 1, k, a - 1, b - 1, c);

    let x1 = interpolate(A, B, u);
    let x2 = interpolate(C, D, u);

    let result = interpolate(x1, x2, v);
    result = interpolate(result, interpolate(gradient(i, j, k + 1, a, b, c - 1),
gradient(i + 1, j, k + 1, a - 1, b, c - 1), u), w);

    return result;
}

function fade(t) {
    return t * t * t * (t * (t * 6 - 15) + 10);
}

function gradient(i, j, k, a, b, c) {
    let h = i * 4096 + j * 64 + k;
    let seed = (h * 119881 + 732453) % 65536;
    let rand = seed / 65536;
    return rand;
}

function interpolate(a, b, t) {
    return a + (b - a) * t;
}

// Attach the function to the object
this.init = init;
this.onBeforeRender = onBeforeRender;

```

A.2 Appendix B · GLM-4.5-Air-Q8_0 Script

```
// Parameters for noise (adjustable in Inspector)
var amplitude = 0.1;    // Displacement strength
var frequency = 1.0;    // Noise density
var speed = 1.0;        // Animation speed

// Private variables
var originalPositions = null;
var clock = new THREE.Clock();
var time = 0;

function start() {
    // Store original positions once
    var geometry = this.object.geometry;
    if (!geometry.attributes.position) return;
    originalPositions = geometry.attributes.position.array.slice();
    // Add vertex colors if not present (optional)
    if (!geometry.attributes.color) {
        var colors = [];
        var color = new THREE.Color(0x4488ff);
        for (var i = 0; i < geometry.attributes.position.count; i++) {
            colors.push(color.r, color.g, color.b);
        }
        geometry.setAttribute('color', new THREE.Float32BufferAttribute(colors, 3));
    }
}

function onBeforeRender() {
    // Skip if no geometry or original positions
    if (!this.object.geometry || !originalPositions) return;
    // Update time
    time += clock.getDelta() * speed;
    var geometry = this.object.geometry;
    var positions = geometry.attributes.position;
    var colors = geometry.attributes.color;
    // Apply noise to each vertex
    for (var i = 0; i < positions.count; i++) {
        var idx = i * 3;
        var x = originalPositions[idx];
        var y = originalPositions[idx + 1];
        var z = originalPositions[idx + 2];
        // Calculate 3D noise coordinates
        var nx = x * frequency;
        var ny = y * frequency;
        var nz = z * frequency + time;
        // Generate noise value (-1 to 1)
        var noise = perlinNoise(nx, ny, nz);
        // Displace vertex along normal
        positions.array[idx] = x + noise * amplitude;
        positions.array[idx + 1] = y + noise * amplitude;
        positions.array[idx + 2] = z + noise * amplitude;
        // Optional: Color modulation
        if (colors) {
            var intensity = (noise + 1) / 2; // Normalize to 0-1
        }
    }
}
```

```

        colors.array[idx] = 0.2 + intensity * 0.8;    // R
        colors.array[idx + 1] = 0.4 + intensity * 0.6; // G
        colors.array[idx + 2] = 1.0;                // B
    }
}
// Update GPU buffers
positions.needsUpdate = true;
if (colors) colors.needsUpdate = true;
// Recalculate normals after displacement
geometry.computeVertexNormals();
}

// 3D Perlin noise implementation
function perlinNoise(x, y, z) {
    // Generate gradients for integer coordinates
    var grad = function(x, y, z) {
        var n = Math.sin(x * 12.9898 + y * 78.233 + z * 37.719) * 43758.5453;
        return (n - Math.floor(n)) * 2 - 1; // -1 to 1
    };
    // Trilinear interpolation
    var lerp = function(a, b, t) {
        return a + (b - a) * t;
    };
    var i = Math.floor(x);
    var j = Math.floor(y);
    var k = Math.floor(z);
    var fx = x - i;
    var fy = y - j;
    var fz = z - k;
    // Smooth gradients
    fx = fx * fx * (3 - 2 * fx);
    fy = fy * fy * (3 - 2 * fy);
    fz = fz * fz * (3 - 2 * fz);
    // Get corner gradients
    var a = grad(i, j, k);
    var b = grad(i + 1, j, k);
    var c = grad(i, j + 1, k);
    var d = grad(i + 1, j + 1, k);
    var e = grad(i, j, k + 1);
    var f = grad(i + 1, j, k + 1);
    var g = grad(i, j + 1, k + 1);
    var h = grad(i + 1, j + 1, k + 1);
    // Interpolate along X
    var x1 = lerp(a, b, fx);
    var x2 = lerp(c, d, fx);
    var x3 = lerp(e, f, fx);
    var x4 = lerp(g, h, fx);
    // Interpolate along Y
    var y1 = lerp(x1, x2, fy);
    var y2 = lerp(x3, x4, fy);
    // Interpolate along Z
    return lerp(y1, y2, fz);
}

```